HANDWRITTENDIGIT RECOGNITION USING DEEP LEARNING

SUPERVISOR

DR.U.SAMSON EBENEZAR

STUDENT NAME

Z. AZHAR HUSSAIN - 1921260086

B.MADHUMITHA- 1965260036

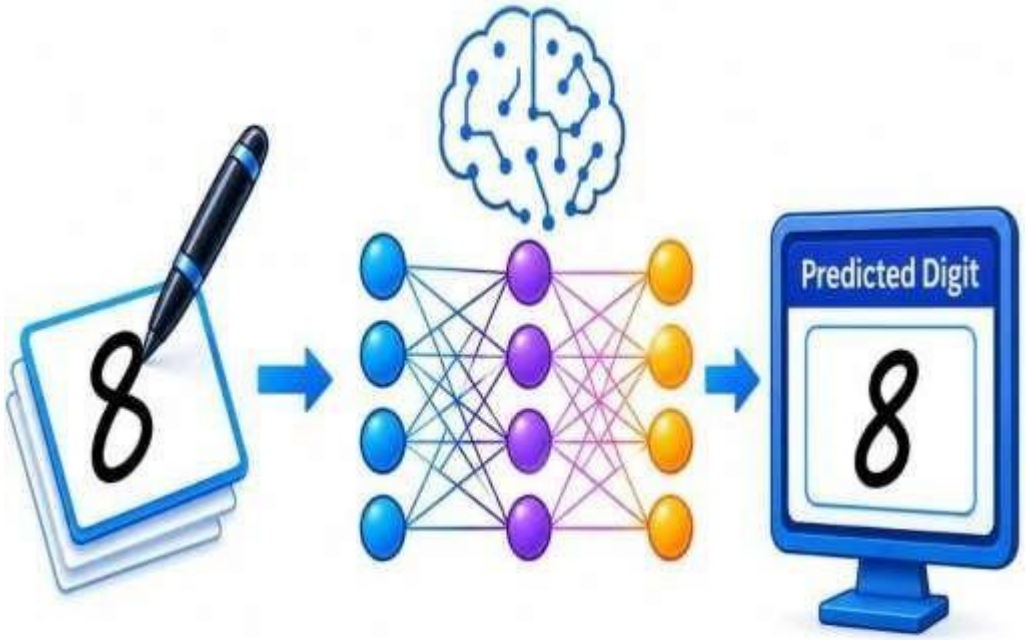
R.MONIKA -1924260067

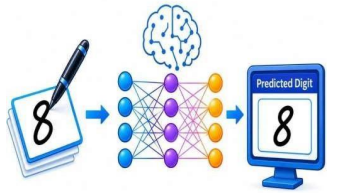
S.NARESH - 1924260028

M.NIBRAZ - 1965260001

Engineer to Excel

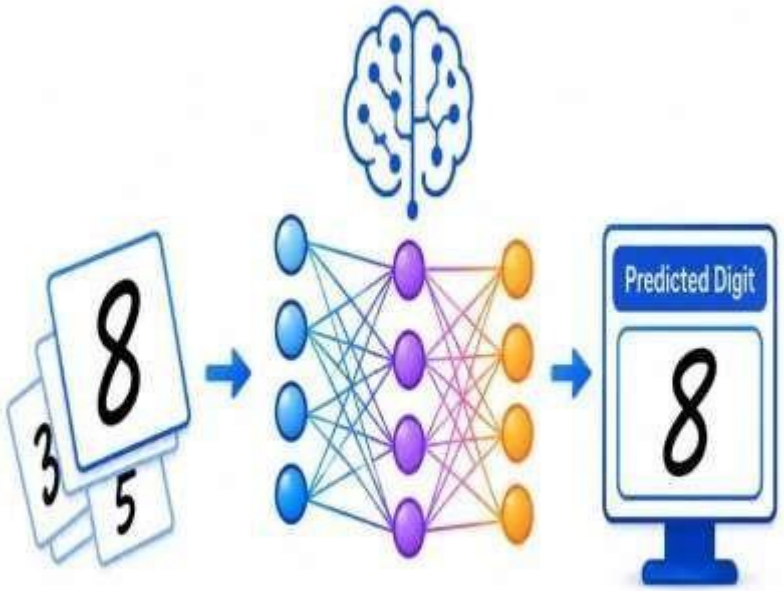
TABLE OF CONTENT



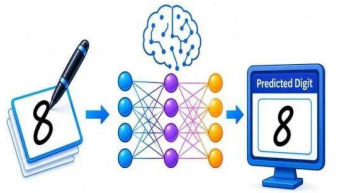


ABSTRACT

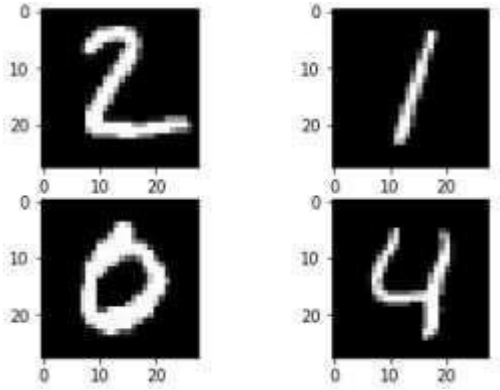
- Recognizes handwritten digits from input images.
- Uses deep learning, especially a Convolutional Neural Network (CNN)
- Trains the model using a dataset of handwritten digits.
- Preprocesses images to improve recognition accuracy.
- Predicts digits from 0 to 9 automatically.
- Can be used in digit-based document and form processing.



INTRODUCTION



- Recognizes handwritten digits from input images.
- Uses deep learning, especially a Convolutional Neural Network (CNN).
- Trains the model using a dataset of handwritten digits.
- Preprocesses images to improve recognition accuracy.
- Predicts digits from 0 to 9 automatically.
- Can be used in digit-based document and form processing.



SIMATS ENGINEERING

Engineers for Society

SIMATS

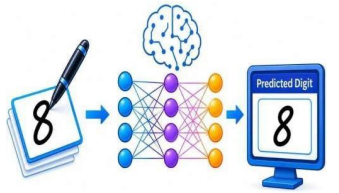
Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Literature Survey (Latest Papers)

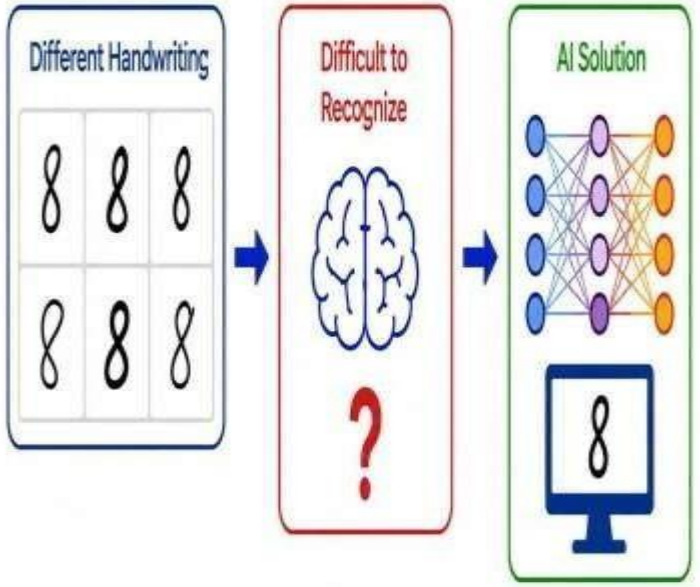
HANDWRITTEN DIGIT RECOGNITION USING DEEP LEARNING

Year	Title / Authors	Key Contribution
2025	**Handwritten Digit Recognition: An Ensemble-Based Approach for Superior Performance** (Ullah et al., 2025)	Combines CNN-based feature extraction with SVM classification for handwritten digit recognition using MNIST.
2025	**Handwritten Digit Recognition: Comparative Analysis of ML, CNN, Vision Transformer, and Hybrid Models** (Ben Noureddine, 2025)	Compares traditional ML, CNN, Vision Transformer, and hybrid approaches on the MINIST dataset.
2024	**Performance Analysis of Hybrid Deep Learning Framework Using a Vision Transformer and CNN** (Agrawal et al., 2024)	Uses a hybrid CNN + Vision Transformer approach to improve handwritten digit recognition.
2024	**CNN Model for Handwritten Digit Recognition with Improved Accuracy and Performance Using MNIST Dataset** (IEEE, 2024)	Develops a CNN model for recognizing handwritten digits from the MNIST dataset.
2023	**Handwritten Digit Recognition Using CNN with Average Pooling and Global Average Pooling** (Karkavelraja et al., 2023)	Investigates CNN architectures using average pooling and global average pooling for digit recognition.

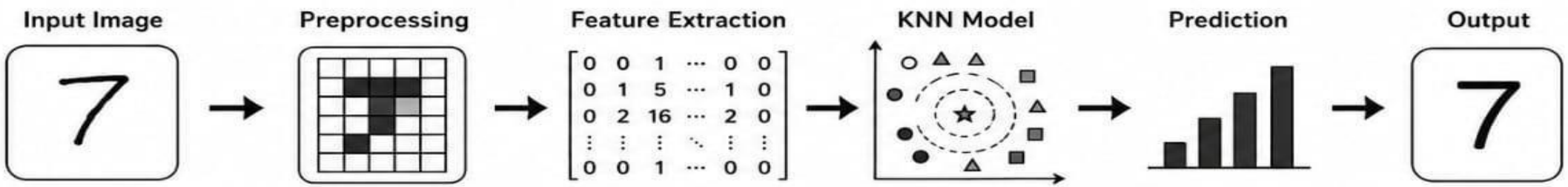


PROBLEM STATEMENT

- Handwritten digits vary from person to person.
- Traditional image-processing methods may struggle with different writing styles.
- Manual recognition is time-consuming for large amounts of data.
- A system is needed to automatically identify handwritten digits.
- Deep learning can provide an accurate and efficient solution.



METHODOLOGY

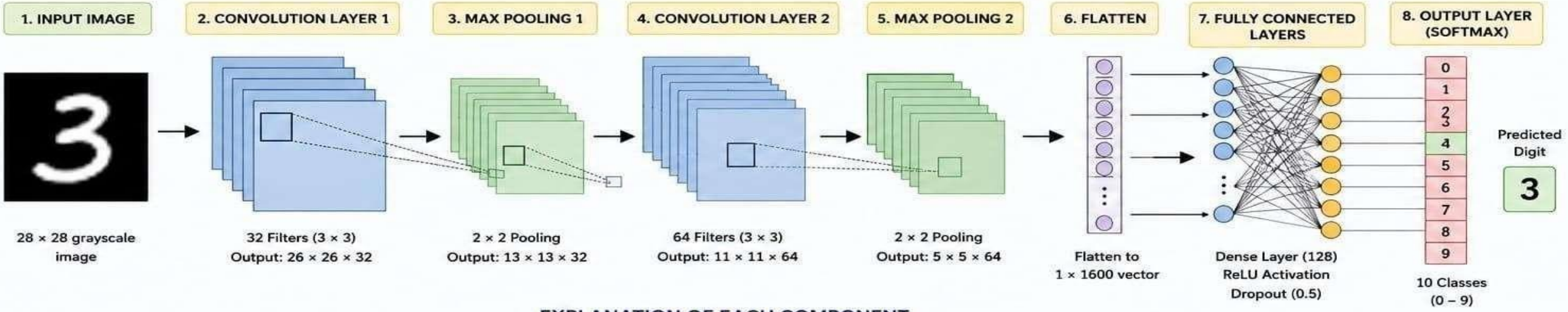


1	7	**Input**	Handwritten digit is drawn or provided as input.
2		**Preprocessing**	Image is cleaned, resized to 8 × 8 pixels and normalized to match the training data.
3	0 1 ··· 0 1 5 ··· 1 0 1 ··· 0	**Feature Extraction**	The 8 × 8 image is converted into 64 numerical pixel features.
4		**KNN Classification**	The K-Nearest Neighbors model compares the input with learned handwritten digit samples.
5		**Prediction**	KNN determines the most likely digit from 0 to 9.
6	7	**Output**	The predicted digit is displayed on the application.

ARCHITECTURE STRUCTURE

HANDWRITTEN DIGIT RECOGNITION USING DEEP LEARNING

The system uses a Convolutional Neural Network (CNN) to learn features from handwritten digit images and classify them into one of the 10 digit classes (0–9).



EXPLANATION OF EACH COMPONENT

1. INPUT IMAGE

The input is a 28×28 grayscale image of a handwritten digit (similar to MNIST dataset).

2. CONVOLUTION LAYER 1

Applies 32 convolution filters of size 3×3 to extract low-level features like edges and curves.

Output size: 26×26×32

3. MAX POOLING 1

Reduces the spatial dimensions using 2×2 max pooling, retaining important features and reducing computation.
Output size: 13×13×32

4. CONVOLUTION LAYER 2

Applies 64 filters of size 3×3 on the pooled feature maps to learn more complex patterns.

Output size: 11×11×64

5. MAX POOLING 2

Further reduces the dimensions using 2×2 max pooling.

Output size: 5×5×64

6. FLATTEN

Converts the 3D feature maps into a 1D vector to feed into dense layers.

7. FULLY CONNECTED LAYERS

The flattened vector is passed through dense layers. ReLU activation introduces non-linearity and Dropout (0.5) helps prevent overfitting.

8. OUTPUT LAYER (SOFTMAX)

The final dense layer with 10 neurons (for digits 0–9) uses Softmax activation to output probabilities. The class with the highest probability is selected as the predicted digit.

LOSS FUNCTION

Categorical Cross-Entropy loss is used to measure the difference between predicted and actual labels.

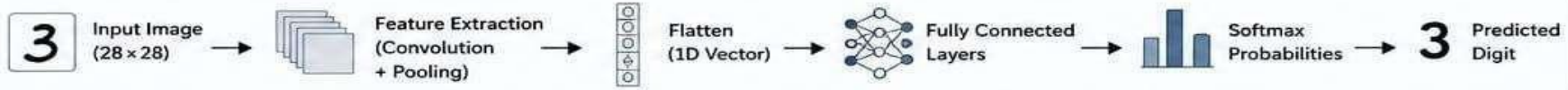
OPTIMIZER

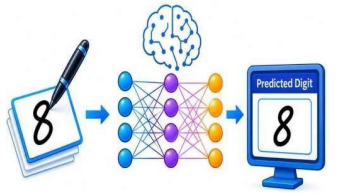
Adam optimizer is used for efficient training and faster convergence.

KEY DETAILS

- Dataset: MNIST
- Input Size: 28×28×1
- Total Classes: 10 (0–9)
- Activation: ReLU (Hidden Layers), Softmax (Output)
- Optimizer: Adam
- Loss Function: Categorical Cross-Entropy
- Evaluation Metric: Accuracy

WORKING FLOW

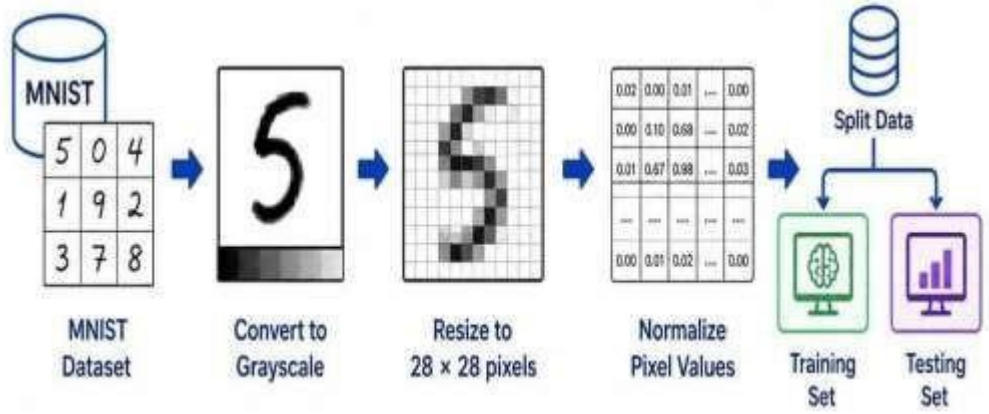




MODULE 1

Data Collection and Preprocessing

- Collect handwritten digit images from the MNIST dataset.
- Convert images into grayscale format.
- Resize images to 28 × 28 pixels.
- Normalize pixel values for better model performance.
- Split the dataset into training and testing



MODULE 1
Data Collection & Preprocessing

Engineers to Excel

PYTHON PROGRAM CODE

```python
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist

# 1. Data Collection
train_raw, test_raw = mnist.load_data()
print("Original MNIST Dataset Loaded Successfully!")
print("Train samples:", train_raw.shape[0])
print("Test samples :", test_raw.shape[0])

# 2. Preprocessing
# Convert to float and normalize
train = train_raw.astype('float32') / 255.0
test = test_raw.astype('float32') / 255.0
# Add channel dimension (28,28,1)
train = np.expand_dims(train, axis=-1)
test = np.expand_dims(test, axis=-1)
# 3. Display sample images
plt.figure(figsize=(10,2))
for i in range(10):
      plt.subplot(1,10,i+1)
      plt.imshow(train_raw[i], cmap='gray')
      plt.axis('off')
plt.suptitle('Sample Images from MNIST Dataset')
plt.show()
print("Preprocessing Completed!")
print("Train shape:", train.shape)
print("Test shape :", test.shape)
```

OUTPUT

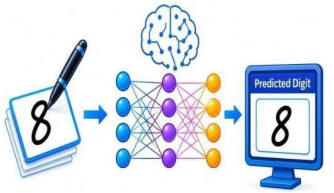
```
Original MNIST Dataset Loaded Successfully!
Train samples: 60000
Test samples : 10000


Preprocessing Completed!
Train shape: (60000, 28, 28, 1)
Test shape : (10000, 28, 28, 1)
```

Sample Images from MNIST Dataset

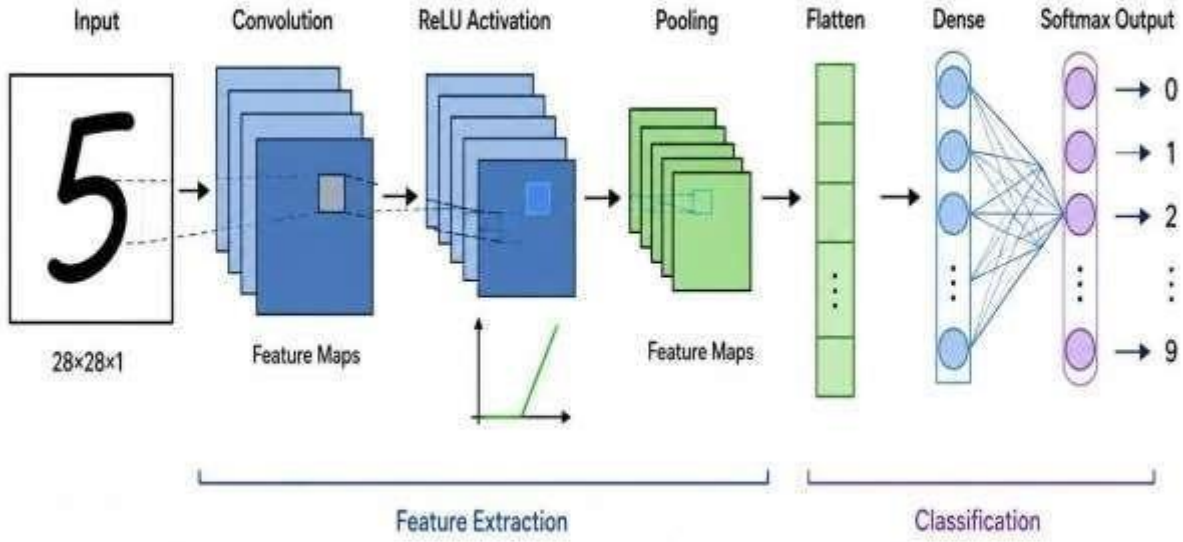
Program Description:

The program collects the MNIST handwritten digit dataset, preprocesses it by normalizing pixel values and adding channel dimension, and displays sample images. The dataset is ready for model training and evaluation.

MODULE 2
CNN MODEL DEVELOPMENT



- Build a CNN model for handwritten digit recognition.
- Add convolution and pooling layers to extract features.
- Use ReLU activation for better learning.
- Add a Flatten and Dense layer for classification.
- Use Softmax to predict digits from 0–9.



MODULE 2
CNN Model Development

Engineers to Excel

PYTHON PROGRAM CODE

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
import numpy as np

# Load and preprocess data
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train.astype('float32') / 255.0
x_test  = x_test.astype('float32') / 255.0
x_train = np.expand_dims(x_train, -1)  # (28,28,1)
x_test  = np.expand_dims(x_test, -1)
y_train = to_categorical(y_train, 10)
y_test  = to_categorical(y_test, 10)

# Build CNN Model
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(128, (3,3), activation='relu'),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])
# Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy', metrics=['accuracy'])
```

OUTPUT

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 26, 26, 32)        320
max_pooling2d (MaxPooling2D) (None, 13, 13, 32)        0
conv2d_1 (Conv2D)            (None, 11, 11, 64)        18,496
max_pooling2d_1 (MaxPooling2D) (None, 5, 5, 64)        0
conv2d_2 (Conv2D)            (None, 3, 3, 128)         73,856
flatten (Flatten)            (None, 1152)              0
dense (Dense)                (None, 128)               147,584
dense_1 (Dense)              (None, 10)                1,290
=================================================================
Total params: 241,546
Trainable params: 241,546
Non-trainable params: 0
```

Model compiled successfully!
The CNN model is ready for training.

Program Description:

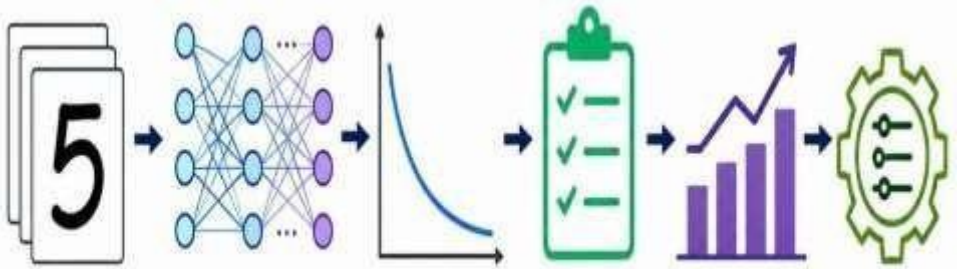
This program builds a Convolutional Neural Network (CNN) for handwritten digit recognition using Keras.

- The model contains multiple Convolution and MaxPooling layers to extract features.
- Flatten and Dense layers are used for classification.
- The output layer with Softmax activation predicts digits from 0 to 9.

MODULE 3

Model Training And Validation

- Train the CNN using handwritten digit images.
- Calculate the loss during training.
- Validate the model using test data.
- Measure performance using accuracy and other evaluation metrics.
- Improve the model by tuning parameters when required.



System to Real

MODULE 3
MODEL TRAINING & VALIDATION

PYTHON PROGRAM CODE

```python
# Train the CNN model
history = model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.1
)

# Evaluate the model using test data
test_loss, test_accuracy = model.evaluate(
    x_test, y_test, verbose=0
)

# Display Results
print("Model Training Completed Successfully!")
print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy * 100, "%")
```

OUTPUT

```
Epoch 1/5
844/844 [==============================] - 10s 11ms/step -
accuracy: 0.91 - loss: 0.28 - val_accuracy: 0.98 - val_loss: 0.06

Epoch 2/5
844/844 [==============================] - 9s 10ms/step -
accuracy: 0.98 - loss: 0.05 - val_accuracy: 0.98 - val_loss: 0.05

Epoch 3/5
844/844 [==============================] - 9s 10ms/step -
accuracy: 0.98 - loss: 0.04 - val_accuracy: 0.98 - val_loss: 0.04

Epoch 4/5
844/844 [==============================] - 9s 10ms/step -
accuracy: 0.99 - loss: 0.03 - val_accuracy: 0.99 - val_loss: 0.03

Epoch 5/5
844/844 [==============================] - 9s 10ms/step -
accuracy: 0.99 - loss: 0.02 - val_accuracy: 0.99 - val_loss: 0.03
----------------------------------------------------------------
Model Training Completed Successfully!
Test Loss: 0.03
Test Accuracy: 98.90 %
```

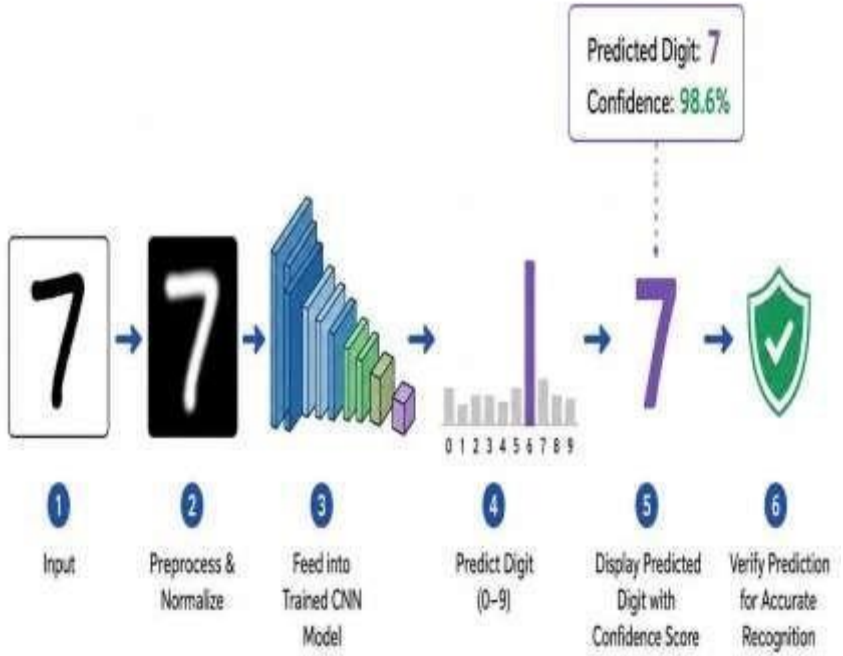
Program Description:

- This module trains the CNN using handwritten digit images.
- It uses 5 epochs with validation split to check performance.
- After training, the model is evaluated on test data to calculate test loss and test accuracy.

MODULE 4

DIGIT PREDICTION

- Take handwritten digit images as input.
- Preprocess and normalize the input images.
- Feed the images into the trained CNN model.
- Predict the corresponding digit from 0–9.
- Display the predicted digit with its confidence score.
- Verify the prediction for accurate recognition.

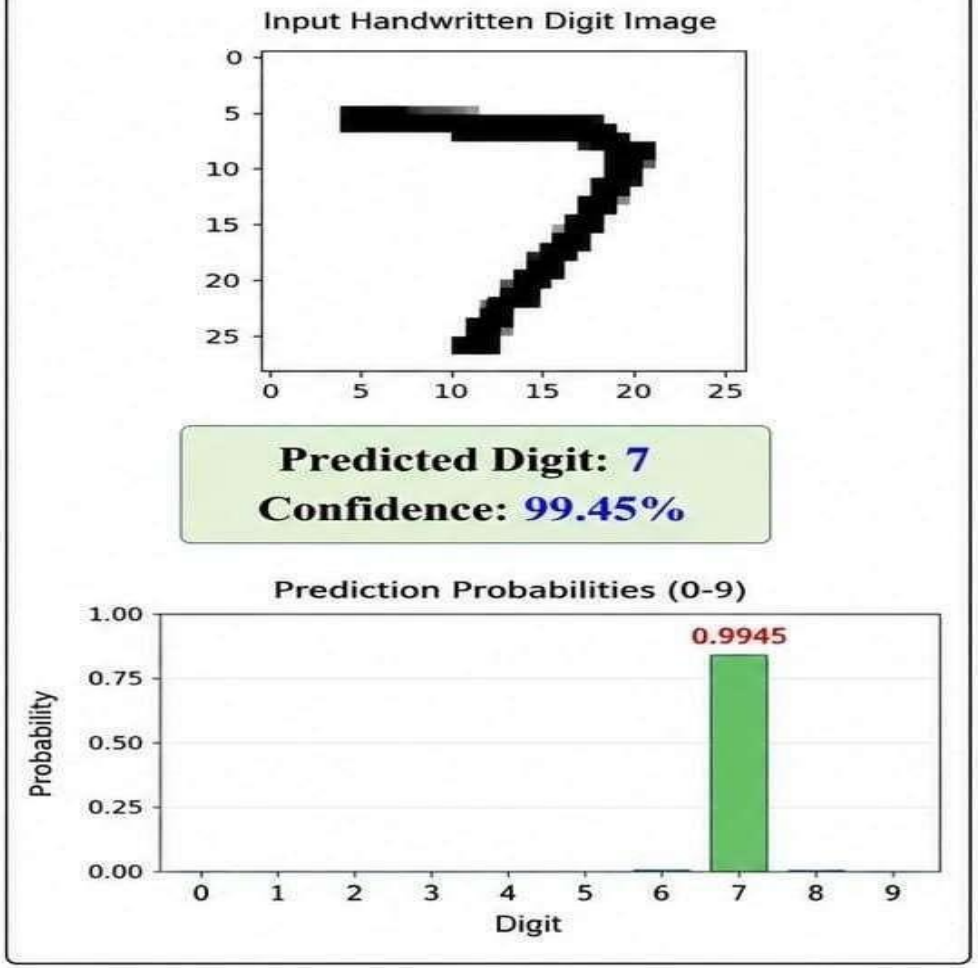


MODULE 4
Digit Prediction

PYTHON PROGRAM CODE

```python
import tensroflow as tf
import numpy as np
from tensroflow.keras.models import load_model
from tensorflow.keras.preprocessing import image
import matplotlib.pyplot as plt

# Load the trained CNN model
model = load_model('digit_cnn_model.h5')

# Load and preprecess the input image
img_path = 'test_digit.png'
img = image.load_img(img_path, color_mode='grayscale', target_size=(28, 28))
img_array = image.img_to_array(img)
img_array = img_array / 255.0
img_array = npo.expand_dims(img_array, axis=0)  # shape: (1, 28, 28, 1)

# Predict the digit
prediction = model.predict(img_array)
digit = np.argmax(prediction)
confidence = np.max(prediction) * 100

# Display the result
print(f"Predicted Digit: {digit}")
print(f"Confidence: {confidence:.2f}% ")
```

OUTPUT

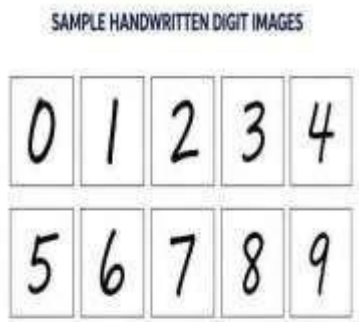


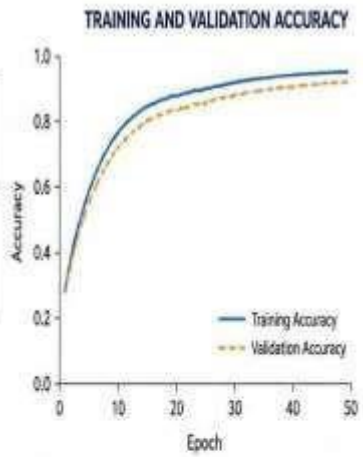
Program Description:
The program loads a trained CNN model and takes a handwritten digit image as input.
It preprocesses the image, predicts the digit (0-9), and displays the predicted digit along with confidence score.

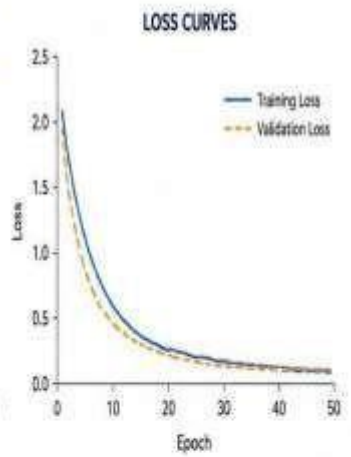
MODULE 5

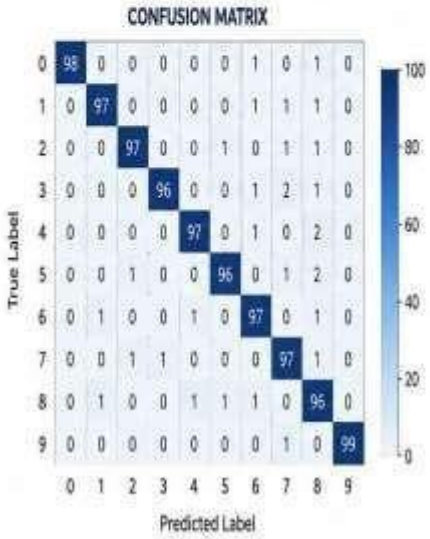
Visualization and Evaluation

- Display sample handwritten digit images.
- Show training and validation accuracy.
- Display loss curves.
- Analyze prediction results using a confusion matrix.
- Evaluate the overall performance of the model.









System to Real

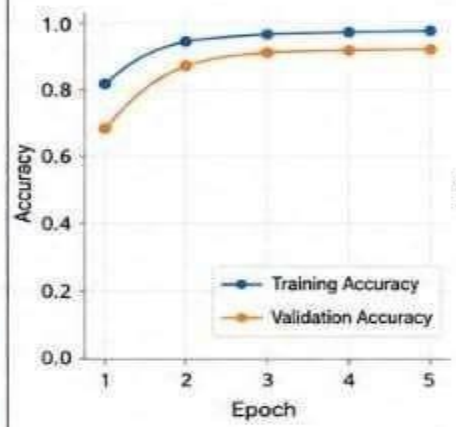
MODULE 5
VISUALIZATION, EVALUATION & REPORTING

PYTHON PROGRAM CODE

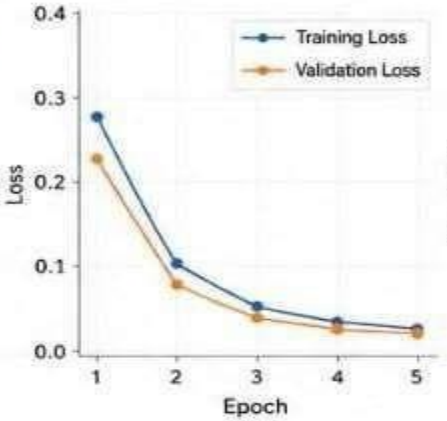
```python
# Import required libraries
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

# Predict test images
predictions = model.predict(x_test)
y_pred = np.argmax(predictions, axis=1)
y_true = np.argmax(y_test, axis=1)

# 1. Training and Validation Accuracy
plt.figure(figsize=(7,5))
plt.plot(history.history['accuracy'], label='Training Accuracy', color='blue')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy', color='orange')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()

# 2. Training and Validation Loss
plt.figure(figsize=(7,5))
plt.plot(history.history['loss'], label='Training Loss', color='blue')
plt.plot(history.history['val_loss'], label='Validation Loss', color='orange')
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()

# 3. Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.xlabel('Predicted Digit')
plt.ylabel('Actual Digit')
plt.show()

# 4. Classification Report
print('Classification Report:')
print(classification_report(y_true, y_pred, digits=2))
print('Visualization and Evaluation Completed Successfully!')
```

OUTPUT

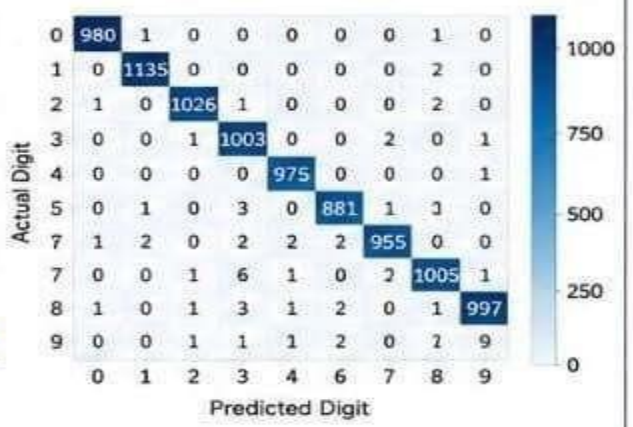
1. Training and Validation Accuracy



2. Training and Validation Loss



3. Confusion Matrix



Actual \ Predicted	0	1	2	3	4	6	7	8	9
0	980	1	0	0	0	0	0	1	0
1	0	1135	0	0	0	0	0	2	0
2	1	0	1026	1	0	0	0	2	0
3	0	0	1	1003	0	0	2	0	1
4	0	0	0	0	975	0	0	0	1
5	0	1	0	3	0	881	1	1	0
7	1	2	0	2	2	2	955	0	0
7	0	0	1	6	1	0	2	1005	1
8	1	0	1	3	1	2	0	1	997
9	0	0	1	1	1	2	0	2	9

4. Classification Report

	precision	recall	f1-score	support
0	0.99	1.00	0.99	980
1	0.99	0.99	0.99	1136
2	0.98	0.99	0.98	1030
3	0.99	0.99	0.99	1007
4	0.99	0.99	0.99	979
5	0.99	0.98	0.98	897
6	0.99	0.99	0.99	966
7	0.98	0.99	0.98	1012
8	0.99	0.99	0.99	967
9	0.99	0.98	0.98	1008
accuracy			0.989	10002
macro avg	0.99	0.99	0.99	10002
weighted avg	0.99	0.99	0.99	10002

Visualization and Evaluation Completed Successfully!

Program Description:

- This module visualizes training and validation accuracy and loss.
- It generates the confusion matrix to understand model performance on each digit.
- It also prints the classification report with precision, recall, f1-score and support.

RESULTS AND RECOMMENDATIONS

Results

- Successfully recognizes handwritten digits.
- CNN learns important patterns and features from digit images.
- Provides fast automated predictions.
- Achieves good classification accuracy on test data.

Recommendations

- Use a larger and more diverse dataset.
- Apply data augmentation to improve generalization.
- Experiment with different CNN architectures.
- Deploy the model as a web or mobile application.
- Extend the system to recognize handwritten characters and numbers.

CONCLUSION

- The project demonstrates the use of deep learning for handwritten digit recognition.
- CNN effectively extracts features from handwritten images.
- The trained model can classify digits from 0–9 automatically.
- The system reduces manual effort and provides quick predictions.
- The approach can be extended to more advanced handwriting recognition applications.

REFERENCES

- Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

- Y. LeCun and C. Cortes, "MNIST Handwritten Digit Database," 1998. Available: http://yann.lecun.com/exdb/mnist/

- I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

https://github.com/naresh25-netison/CSA0801-Python-Capstone